

Project 4

ADC and addressable RGB LEDs

EELE 465

GitHub Classroom assignment This project is a group assignment. If you need to change your team from last time, let me know and I will get that all sorted out. Accept the new [GitHub Classroom assignment](#), which will create a new repository for your team. Each student must accept the assignment.

Project overview

Project Goal Read an analog temperature sensor with an ADC, display the temperature data over UART, use a slide potentiometer to control the speed of the LED patterns, and control addressable RGB LEDs.

The primary purpose of this project is to use the MSP430's ADC to sample an analog temperature sensor ([LMT87LPM](#)) and a slide potentiometer. The slide potentiometer will be used to control the speed of the LED patterns (that is, the voltage value you read with the ADC will be mapped to the speed of the LED patterns). You will display the temperature data and the RTC's date and time on your computer via UART; I provide a GUI skeleton that you can use for sending and display values via UART. As a fun little addon, you will control [a strip of addressable WS2812B LEDs](#) (commonly known as “neopixels”, which is Adafruit's trade name for addressable RGB LEDs).

This project builds on project 3. You will keep the LED pattern and RTC functionality from project 3, except you won't need to display the RTC registers on the LED bar anymore.

Outcomes

After this project, you should be able to:

- sample analog signals using the MSP430's ADC module.
- calculate moving averages.

Project requirements



From now on, it is assumed that you have a heartbeat LED and reset button; those are non-negotiable, but aren't explicitly worth any extra points since you've already implemented them.

Temperature sensor

Requirement 1 The system must sample the LMT87's voltage output. 12 pts

Specification 1.1 The voltage must be sampled at least every 0.5 s.

Requirement 2 The system must convert the temperature sensor's voltage into degrees Celsius. 7 pts

Specification 2.1 The temperature must be within 1.5 °C of the "actual" temperature (don't stress about this too much, just make sure the temperature seems reasonable...).

Requirement 3 The system must calculate a moving average of the temperature values. 7 pts

Specification 3.1 The default moving average window size is $n = 5$.

Requirement 4 The temperature must be displayed to the user via UART. 5 pts

Requirement 5 The user can change the moving average window size via UART. 5 pts

Specification 5.1 The minimum window size is $n = 1$.

Specification 5.2 The maximum window size is $n = 16$.

Slide potentiometer

Requirement 6 The system must sample the voltage at the slide potentiometer's wiper. 12 pts

Specification 6.1 The voltage must be sampled at least every 10 ms.

Requirement 7 The position of the slide potentiometer must control the speed of the LED patterns. 5 pts

Specification 7.1 All LED patterns will operate at the same speed.

Specification 7.2 The minimum LED pattern period is 0.1 s.

Specification 7.3 The maximum LED pattern period is 1.0 s.

Specification 7.4 The slide potentiometer's position (i.e., the ADC's range) must be broken up into at least 16 intervals; in other words, there must be at least 16 different pattern periods.

WS2812B LED strip

Requirement 8 The system can individually turn on each LED. 6 pts

Requirement 9 The color of the LEDs can be changed by pressing a button. 5 pts

Specification 9.1 The LEDs must be able to be at least red, green, blue, and white.

Requirement 10 The position of the slide potentiometer must be mapped to the LED strip. 8 pts (A)



Since there are less LEDs than there are ADC values (10 vs. 4096), you can't just split the ADC values into 10 bins—that resolution would be atrocious! Instead, you need to vary the brightness of each LED (in sequence) to indicate the potentiometer position with finer resolution.

Control and display GUI

I've provided a GUI in your repository that you can use to communicate with your MSP430 via UART. The intent is that the GUI makes the user experience / user interface for setting the RTC's date and the moving average window size a bit better than typing in a terminal while text from the MSP430 is streaming in the terminal. I have provided a skeleton that you will need to flesh out.

The GUI is written in Python and only uses packages from the standard library. If you're on Windows, see [Python's "Using Python on Windows" documentation](#) for information on how to install Python. If you're on macOS or Linux, Python likely already exists or you can install it using your package manager.

See `gui/README.md` for documentation on how to run the GUI program.

Requirement 11 The GUI must display the LMT87's temperature in degrees Celsius.

2 pts (100)

Requirement 12 The GUI must display the RTC's date and time.

2 pts (100)

Requirement 13 The user must be able to use the GUI to set the RTC's date and time.

2 pts (100)

Requirement 14 The user must be able to use the GUI to set the temperature's moving average window size.

2 pts (100)

Project 3 leftovers

If you didn't complete any requirements in project 3 (e.g., multiplexed LED bar, UART, etc.), you can get 50% of those points back if you complete those requirements in this project. If that is the case, please let us know during your demo so we can update your project 3 grading rubric accordingly.

Development Process

Git Procedure

Use the same general git procedure as last time. The generic procedure is documented in your repository and on Canvas.

- Start with a planning branch.
- Create branches for each feature.
- Use pull requests to merge the branches once they are done. Partners should review the pull requests and perform a code review.

Planning



You must create a planning branch and complete the workload distribution, architecture sketch, pseudocode, and preliminary circuit/block diagram before writing code. You will create a pull request for the planning branch, which needs to be approved and merged before creating other branches for writing code. You will lose points if you don't follow this procedure!

Workload Distribution

Fill out the workload distribution form located at docs/planning/workload-distribution.md. You may want to sketch out your software architecture before making final decisions on how to split the work.

Software Architecture

Create a high-level software architecture diagram. It can be thought of as a block diagram showing what software components will be used (e.g., keypad, i2c driver, pwm, rgb led module, etc.) and how they interact with and relate to each other. Some formal diagram types that are used in industry include [C4 component diagrams](#), [UML class diagrams](#), [UML component diagrams](#), [UML package diagrams](#); ad-hoc “boxes and lines” block diagrams are also common and totally acceptable for this class.

Pseudocode or Flowcharts

Before writing any code, you should write pseudocode to plan out what your code needs to do. Granted, if the code truly is trivial, you don’t need to write pseudocode. However, when in doubt, err on the side of writing pseudocode.

For the initial planning branch, you don’t need to write pseudocode for everything. Just write pseudocode or create a flowchart for the high-level operation of the main loop.

For each individual development branch, I expect you to write pseudocode before writing any non-trivial code.



Pseudocode should go in docs/planning/. Flowcharts should go in docs/assets/, but the images should be included in a markdown file in docs/planning/.

Circuit Diagram

Create a preliminary circuit diagram for your system before connecting your hardware.

Tips

- Test your hardware *before* testing your software. If you test your software without first knowing that your hardware works, you won’t know if your software is broken or your hardware is broken...or both...
- To the extent possible, test everything independently rather than testing a module’s functionality during integration.
- Test the ADC with constant, known voltages from the AD2 or another power supply.

Demo

1. Introduction

- Introduce the project at a high level. Don't assume the instructor or TA knows about the project. Focus on the goal of the project.
- Use a software architecture diagram to discuss how the software was *designed*.
- Use a high-level flowcharts to help describe the system's functionality.
- Use a circuit diagram to explain how the circuit was constructed.

2. Demonstration

- Demonstrate the full system working. You may choose the order in which you would like to present functionality.
- Be sure to demonstrate that every specification was met.
- The partner that was primarily responsible for a subsystem/requirement/specification should present that functionality.
- If you completed any extra credit, demonstrate that functionality.